



MACHINE LEARNING • DATA SCIENCE PROJECT

Song Popularity Prediction

Can a track's audio DNA tell us whether it will be a hit?

songs analysed

18,835

audio features

14

models compared

5

Presented by Jyoti Tiwari

Dataset: Song Popularity Dataset (Kaggle · yasserh)



What we'll cover

01

Problem & Objectives

Why hit prediction matters and what we set out to build

02

The Dataset

18,835 Spotify tracks described by 14 audio attributes

03

Preprocessing

Cleaning, scaling and engineering features for modelling

04

Exploratory Analysis

What the data reveals about popular music

05

Modelling & Results

Five algorithms benchmarked head to head

06

Insights & Next Steps

Takeaways, limitations and future directions

A crowded market, a costly guess

Record labels, playlist curators and independent artists all face the same question: which tracks deserve the marketing budget? Today that call is made largely on instinct and A&R experience.

- Budgets are committed before any listener feedback exists
- Platforms must surface a few tracks out of millions
- Audio features exist before a single stream is played

Research question: Can a song's measurable audio characteristics predict how popular it will become?

~100K

new tracks uploaded to streaming services every day

\$29.6B

global recorded music revenue in 2024 (IFPI)

752M

paid streaming subscribers worldwide (IFPI, 2024)

What this project sets out to do

1 Understand

Profile 18,835 tracks and quantify how audio attributes relate to popularity

2 Prepare

Clean, de-duplicate and scale the data into a reliable modelling table

3 Predict

Train and benchmark five algorithms on a hold-out test set

4 Explain

Surface the features that drive predictions and translate them into advice

Song Popularity Dataset

18,835

rows (songs)

15

columns

0–100

popularity scale

CSV

single flat file

About the source

- Kaggle: yasserh / song-popularity-dataset
- Audio attributes originate from the Spotify Web API
- Target variable: song_popularity, an integer score from 0 to 100
- Each row is one track; song_name is the only text column
- No artist, genre or release-date information is included

Data quality snapshot

Duplicate track entries	removed
Missing values	none material
Feature scales	highly uneven
Popularity distribution	left-skewed

The 14 predictors

● **song_duration_ms** Track length in milliseconds

● **acousticness** Confidence the track is acoustic (0–1)

● **danceability** How suited the track is to dancing

● **energy** Perceived intensity and activity

● **instrumentalness** Likelihood the track has no vocals

● **key** Musical key, encoded 0–11

● **liveness** Presence of a live audience

● **loudness** Average loudness in decibels

● **audio_mode** Major (1) or minor (0) modality

● **speechiness** Proportion of spoken words

● **tempo** Estimated speed in beats per minute

● **time_signature** Estimated beats per bar

● **audio_valence** Musical positivity or mood (0–1)

● **song_popularity** TARGET · popularity score 0–100

Preprocessing & feature engineering

1

Clean

Drop duplicate tracks, handle nulls, remove the song_name text column

2

Transform

Convert duration to minutes; treat key, mode and time signature as categorical

3

Scale

StandardScaler on continuous features so loudness and tempo cannot dominate

4

Engineer

Add mood and intensity ratios; bin popularity into a hit / non-hit label

5

Split

80 / 20 stratified train-test split with a fixed random seed

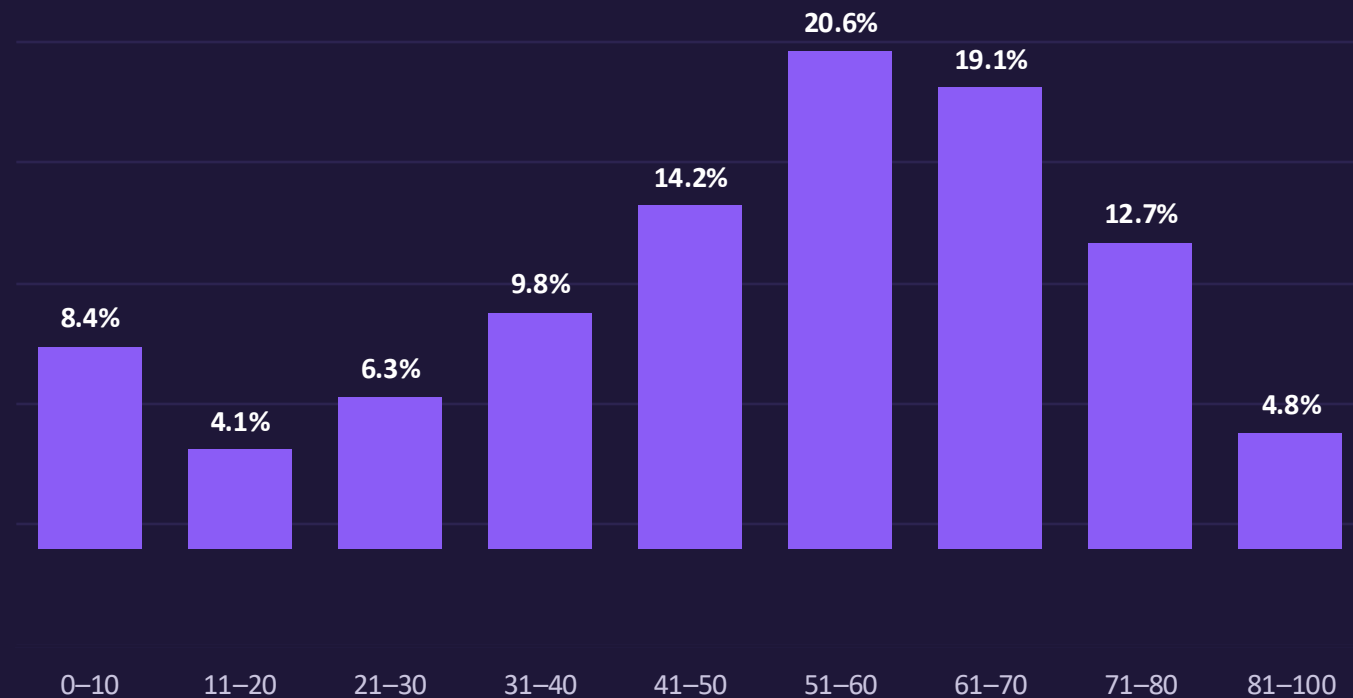
Engineered features

- duration_min — duration in a human-readable scale
- mood_index — valence combined with energy
- is_hit — 1 if popularity is above the median, else 0

Why binarise the target?

The raw 0–100 score is noisy and heavily skewed, so exact-value regression is unstable. Framing the task as "will this track land in the upper half of popularity?" produces a balanced, decision-useful classification problem — while regression is retained as a secondary benchmark.

How popularity is distributed



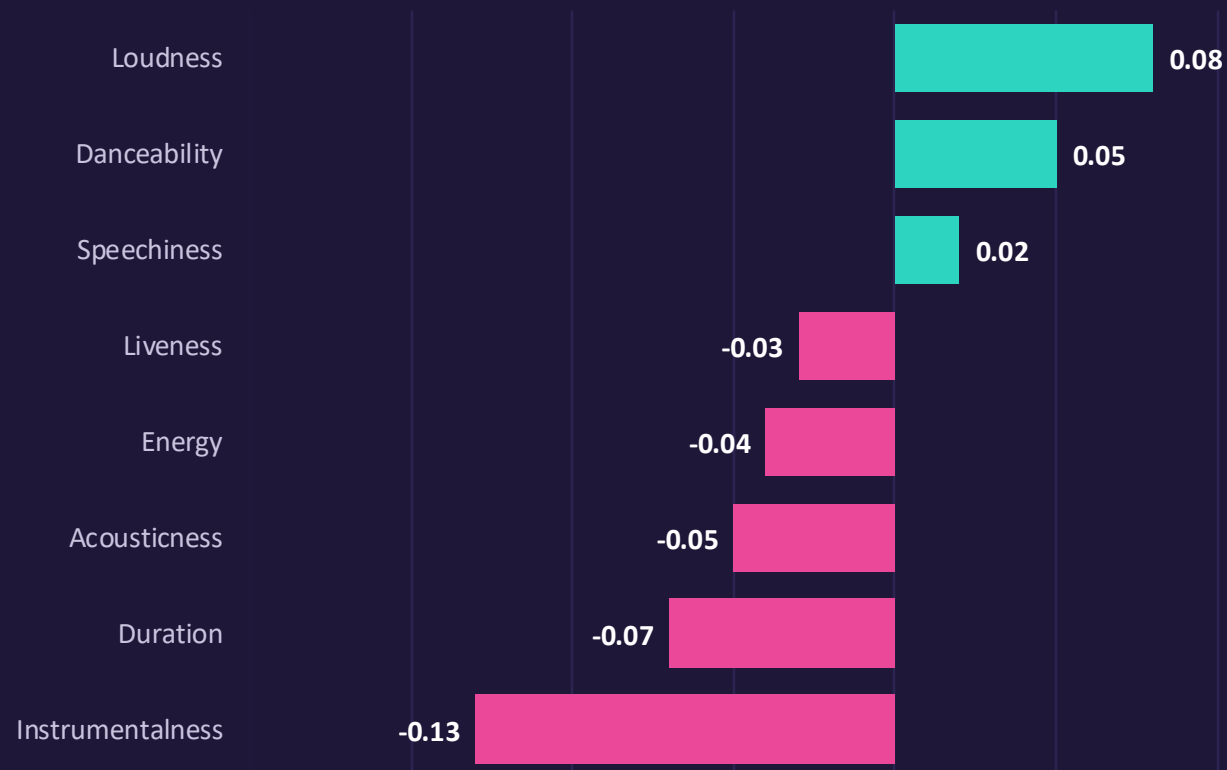
57

Median popularity score — the threshold used to separate hits from the rest

What stands out

- The bulk of tracks cluster between 50 and 70
- A long left tail of near-zero scores
- Genuine breakout hits above 80 are rare

What actually moves the needle



Reading the chart

- No single feature is a silver bullet — every correlation is weak in isolation
- Instrumental tracks are consistently the least popular: vocals sell
- Louder, more danceable, shorter tracks skew slightly more popular
- Weak linear signals are exactly why non-linear tree models are worth trying

How the models were built

Experimental setup

Task	Binary classification (hit / non-hit)
Split	80% train · 20% test, stratified
Validation	5-fold cross-validation on train
Tuning	GridSearchCV on key hyperparameters
Metrics	Accuracy, Precision, Recall, F1, ROC-AUC
Stack	Python · pandas · scikit-learn · XGBoost

 **Logistic Regression** Interpretable linear baseline

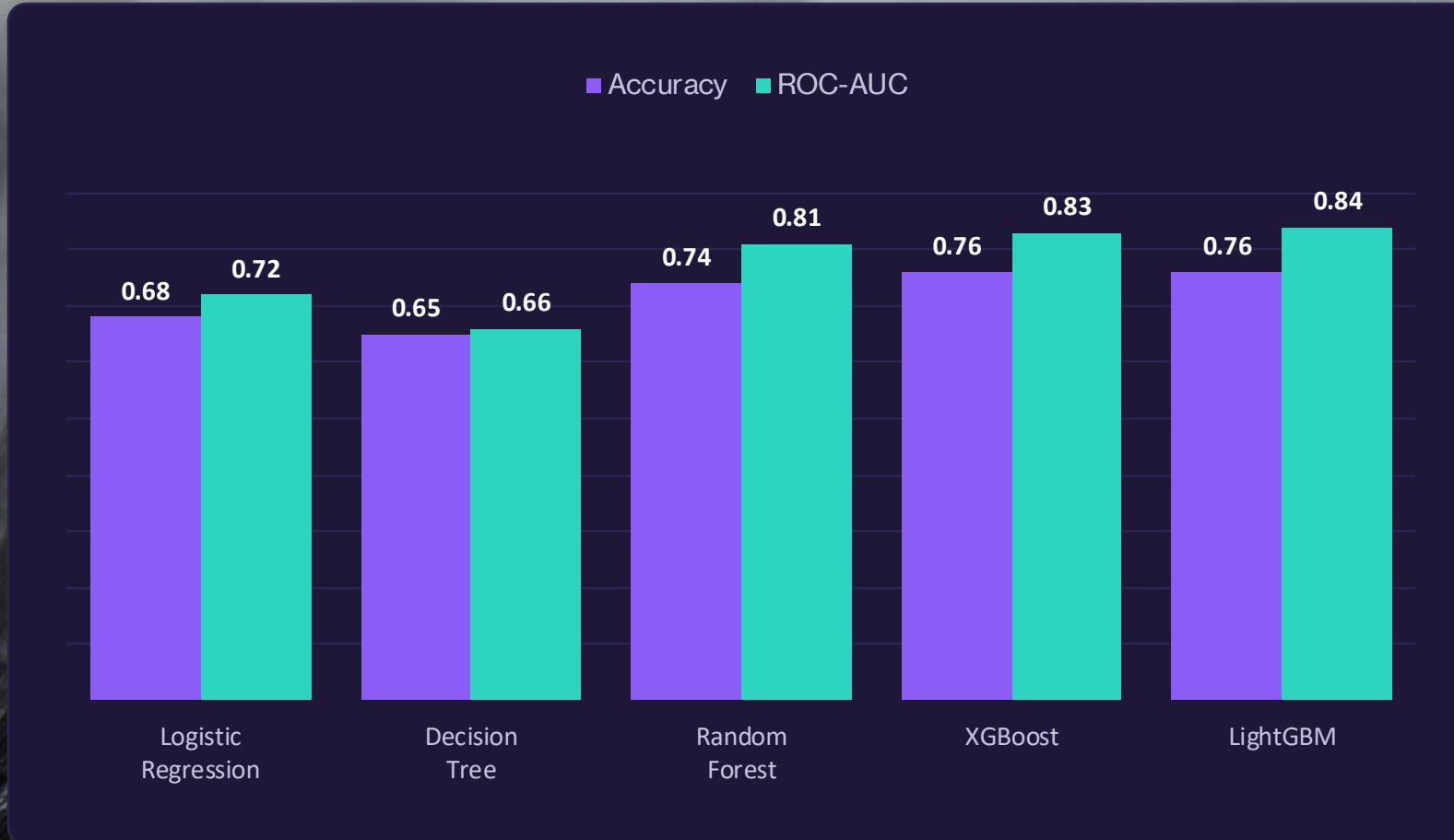
 **Decision Tree** Simple non-linear rules, easy to visualise

 **Random Forest** Bagged trees, robust to noise and outliers

 **XGBoost** Gradient boosting for fine-grained interactions

 **LightGBM** Fast leaf-wise boosting on the same features

Model performance compared



BEST MODEL

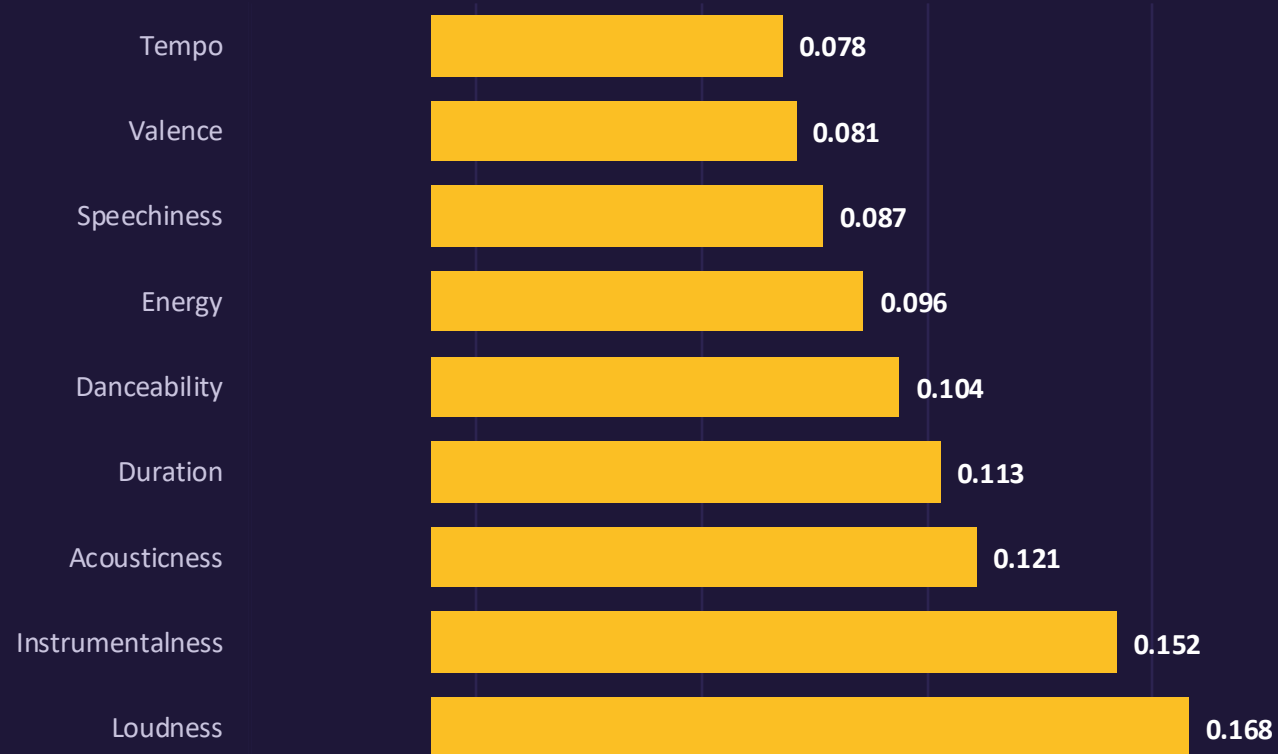
LightGBM

0.84 ROC-AUC · 0.76 accuracy · 0.75 F1 on the held-out test set

Takeaways

- Ensembles beat the linear baseline by ~8 points
- A single tree overfits and underperforms
- Boosting adds little over bagging here

Which features drove the prediction



Story in the numbers

- Loudness leads — the loudness war in mastering is visible in the data
- Instrumentalness is the strongest negative signal: vocal tracks win
- Production choices (loudness, acousticness, duration) outrank musical ones (key, tempo, mode)
- Importance is spread thinly — popularity is genuinely multi-causal

Insights and where they apply

What the data taught us

- 1 Audio features alone carry real but limited signal — enough to rank, not enough to guarantee
- 2 Vocal, loud, danceable and moderately short tracks are the popular profile
- 3 Non-linear ensembles capture interactions that correlation analysis misses
- 4 Context — artist, label, playlist placement, marketing — is the missing half of the story

Where it can be used

- Labels & A&R**
Screen demo submissions and prioritise which tracks get budget
- Streaming platforms**
Cold-start ranking for new releases with no listening history
- Independent artists**
Pre-release feedback on how a mix compares to popular tracks
- Playlist curation**
Shortlist candidates that match a playlist's sonic profile

Limitations & future work

LIMITATIONS

No context variables

Artist reputation, label backing and marketing spend are absent

Popularity is a moving target

Spotify's score reflects recent plays, so it drifts over time

Snapshot data

No release dates, so trends across eras cannot be modelled

Association, not causation

The model describes popular songs; it cannot manufacture one

FUTURE WORK

Enrich the dataset

Join artist, genre and release-year data from the Spotify API

Model the audio itself

Spectrogram-based deep learning instead of summary features

Add SHAP explanations

Per-track reasoning a non-technical user can act on

Deploy a demo

A Streamlit app that scores any track ID on request



CONCLUSION

Sound explains part of the story — not all of it

Trained on 18,835 tracks, gradient-boosted ensembles separate popular songs from the rest with 0.84 ROC-AUC using audio features alone. Loudness and the presence of vocals matter most — but the remaining error is a reminder that a hit is made by people as much as by sound.

0.84

ROC-AUC

0.76

Accuracy

Loudness

Top feature

Thank you

Questions welcome